

RISC-V SECURE IoT GATEWAY SoC

PROJECT ABSTRACT

RISC-V Based SoC with I2C, SPI, DMA and AES-128 Hardware Security

1. Problem Statement

Modern IoT systems collect data from sensors such as temperature, humidity, pressure, motion, and other monitoring devices. A gateway must acquire this information, move it into memory, perform basic processing, and protect sensitive payloads before they are transmitted to a host or higher-level system. When all of these operations are performed by a general-purpose processor, the processor must repeatedly service peripheral transactions, copy data between buffers, and execute cryptographic operations in software. This can increase processor utilization, processing latency, and the time required to complete a complete sensor-data transaction.

The proposed project addresses this problem by designing and verifying a RISC-V based System-on-Chip (SoC) that combines standard embedded-system components with application-specific hardware IPs. The gateway uses I2C and SPI controllers for sensor communication, a DMA controller for block data movement, and an AES-128 accelerator for hardware encryption. The RISC-V processor remains responsible for system initialization, configuration, sequencing, decision making, and lightweight software processing, while the dedicated hardware blocks perform repetitive communication, data-transfer, and cryptographic tasks.

The central application is secure sensor-data acquisition. A sensor connected through I2C or SPI produces a measurement. The processor configures the corresponding communication controller and DMA transfer. The acquired payload is moved into Data Memory by DMA, reducing the need for the CPU to copy every data word. The RISC-V then configures the AES-128 accelerator, which encrypts the sensor payload using a programmed 128-bit key. The encrypted data is stored in memory and can subsequently be transmitted through UART to a host system for monitoring or further processing.

2. Proposed SoC Architecture

The SoC is organized around the RISC-V processor as the central control unit. Instruction Memory stores the application firmware, while Data Memory stores sensor payloads, DMA buffers, encryption input/output, and software data. The processor and memory-mapped IPs are connected through a system bus interconnect. For this project, AXI4 is selected as the primary interconnect so that the processor can access memories and peripheral registers through a consistent memory-mapped interface.

Stage	Function in the proposed gateway
1. Sensor Interface	I2C/SPI acquires measurements from external IoT sensors.
2. DMA Transfer	DMA moves acquired data from the peripheral/data path into Data Memory.
3. Data Buffer	Data Memory holds the raw sensor payload before security processing.
4. AES-128	Hardware AES encrypts the selected 128-bit data block.
5. Secure Buffer	Ciphertext is stored in Data Memory for subsequent transmission.
6. UART Output	Encrypted data and system status are sent to a host/debug interface.

The architecture therefore creates a clear separation of responsibilities. RISC-V performs control and software operations; I2C and SPI perform sensor communication; DMA performs bulk data movement; AES-128 performs

cryptographic acceleration; and UART, Timer, and GPIO provide monitoring and control support. This separation makes the SoC modular and allows each IP to be verified independently before full-system integration.

At the system level, the data path is designed to minimize unnecessary processor involvement. The RISC-V processor first initializes the communication interfaces, DMA, AES accelerator, UART, Timer, and GPIO. It then selects the required sensor interface and configures the transaction. Once the sensor data becomes available, DMA performs the programmed transfer into a memory buffer. The processor can monitor the DMA status and continue other control operations instead of copying each received byte or word.

After acquisition, the processor provides the encryption key and input data to the AES-128 accelerator through its memory-mapped control interface. AES processes the 128-bit plaintext block and produces a 128-bit ciphertext block. The encrypted result is retained in the secure data buffer. A subsequent DMA transfer can move the encrypted data toward the UART output path, allowing the host to receive protected sensor information. Timer measurements can be used to evaluate the latency of the acquisition, transfer, encryption, and transmission stages.

3. Selected IPs

Selected IP	Purpose and contribution
1. I2C Controller	Provides master-mode communication with low-speed IoT sensors. It handles START/STOP conditions, device addressing, read/write transactions, ACK/NACK handling, clock generation, and transfer status.
2. SPI Controller	Provides a higher-speed serial interface for sensors or external peripherals. It supports programmable clocking, chip-select control, transmit/receive data, and configurable SPI timing mode.
3. DMA Controller	Transfers blocks of sensor data between peripheral/data buffers and Data Memory with reduced CPU involvement. The minimum control set includes source address, destination address, transfer length, START, BUSY, DONE, and ERROR.
4. AES-128 Accelerator	Provides hardware acceleration for AES-128 encryption. It contains a 128-bit key interface, 128-bit plaintext input, 128-bit ciphertext output, START control, and BUSY/DONE status.